

Import libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
    precision_recall_curve,
    roc_curve
)

import shap

import warnings
warnings.filterwarnings("ignore")
```

Load the data

```
df = pd.read_csv("/content/creditcard.csv")

df.head()

{"type": "dataframe", "variable_name": "df"}

df.shape

(284807, 31)
```

Inspect the data

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   Time        284807 non-null float64
1   V1          284807 non-null float64
2   V2          284807 non-null float64
3   V3          284807 non-null float64
4   V4          284807 non-null float64
5   V5          284807 non-null float64
6   V6          284807 non-null float64
7   V7          284807 non-null float64
8   V8          284807 non-null float64
9   V9          284807 non-null float64
10  V10         284807 non-null float64
11  V11         284807 non-null float64
12  V12         284807 non-null float64
13  V13         284807 non-null float64
14  V14         284807 non-null float64
15  V15         284807 non-null float64
16  V16         284807 non-null float64
17  V17         284807 non-null float64
18  V18         284807 non-null float64
19  V19         284807 non-null float64
20  V20         284807 non-null float64
21  V21         284807 non-null float64
22  V22         284807 non-null float64
23  V23         284807 non-null float64
24  V24         284807 non-null float64
25  V25         284807 non-null float64
26  V26         284807 non-null float64
27  V27         284807 non-null float64
28  V28         284807 non-null float64
29  Amount      284807 non-null float64
30  Class       284807 non-null int64  
dtypes: float64(30), int64(1)
memory usage: 67.4 MB

df.describe()

{"type": "dataframe"}

df.columns
```

```
Index(['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9',  
      'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19',  
      'V20', 'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28',  
      'Amount',  
      'Class'],  
      dtype='object')
```

```
df.isnull().sum()
```

```
Time      0  
V1        0  
V2        0  
V3        0  
V4        0  
V5        0  
V6        0  
V7        0  
V8        0  
V9        0  
V10       0  
V11       0  
V12       0  
V13       0  
V14       0  
V15       0  
V16       0  
V17       0  
V18       0  
V19       0  
V20       0  
V21       0  
V22       0  
V23       0  
V24       0  
V25       0  
V26       0  
V27       0  
V28       0  
Amount    0  
Class     0  
dtype: int64
```

```
df.duplicated().sum()
```

```
np.int64(1081)
```

```
quality_summary = pd.DataFrame(  
    "Missing Values": df.isnull().sum(),
```

```

    "Unique Values": df.nunique()
})

quality_summary

{"summary": "{\n  \"name\": \"quality_summary\",\n  \"rows\": 31,\n  \"fields\": [\n    {\n      \"column\": \"Missing Values\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 0,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Unique Values\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 69124,\n        \"min\": 2,\n        \"max\": 275663,\n        \"num_unique_values\": 4,\n        \"samples\": [\n          275663\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}", "type": "dataframe", "variable_name": "quality_summary"}

```

Understand the target

```

df["Class"].value_counts()

Class
0    284315
1     492
Name: count, dtype: int64

class_percentage = (
    df["Class"]
    .value_counts(normalize=True)
    .mul(100)
)

class_percentage

Class
0    99.827251
1     0.172749
Name: proportion, dtype: float64

```

Visualize class imbalance

```

plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,

```

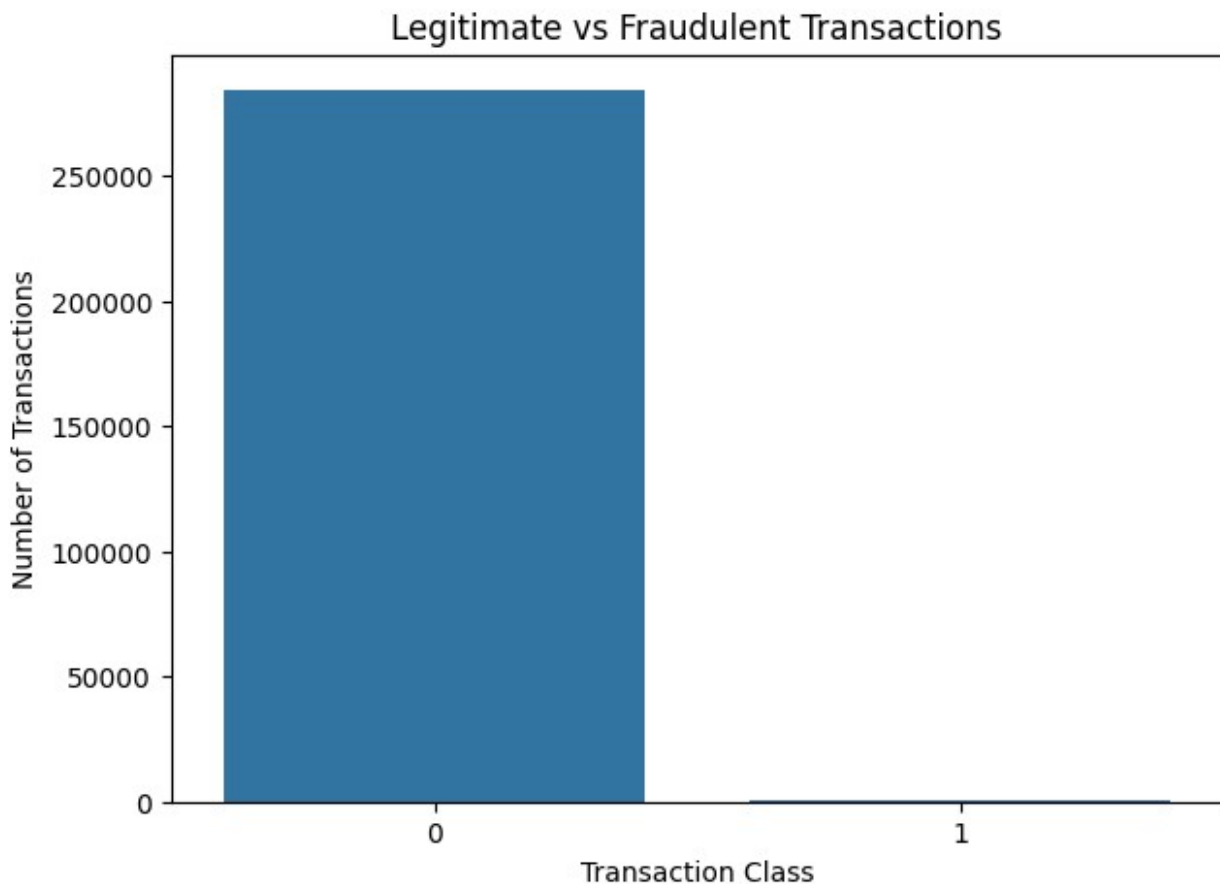
```

    x="Class"
)

plt.title("Legitimate vs Fraudulent Transactions")
plt.xlabel("Transaction Class")
plt.ylabel("Number of Transactions")

plt.show()

```



Analyze transaction amount

```

df.groupby("Class")["Amount"].describe()

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 2,\n  \"fields\": [\n    {\n      \"column\": \"Class\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          1,\n          0\n        ],\n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      },\n      \"count\": 1,\n      \"properties\": {\n        \"dtype\": \"number\",

```

```

{"number": 492.0, "std": 200693.16795670948, "min": 492.0, "max": 284315.0, "num_unique_values": 2, "samples": [492.0, 284315.0], "semantic_type": "", "description": ""}, {"column": "mean", "dtype": "number", "std": 23.985273241884894, "min": 88.29102242231328, "max": 122.21132113821139, "num_unique_values": 2, "samples": [88.29102242231328, 122.21132113821139], "semantic_type": "", "description": ""}, {"column": "properties", "dtype": "number", "std": 4.651487050375273, "min": 250.10509222586853, "max": 256.68328829771207, "num_unique_values": 2, "samples": [250.10509222586853, 256.68328829771207], "semantic_type": "", "description": ""}, {"column": "properties", "dtype": "number", "std": 0.0, "min": 0.0, "max": 0.0, "num_unique_values": 1, "samples": [0.0], "semantic_type": "", "description": ""}, {"column": "25%", "dtype": "number", "std": 3.288046532517446, "min": 1.0, "max": 5.65, "num_unique_values": 2, "samples": [1.0, 5.65], "semantic_type": "", "description": ""}, {"column": "50%", "dtype": "number", "std": 9.015611460128481, "min": 9.25, "max": 22.0, "num_unique_values": 2, "samples": [9.25, 22.0], "semantic_type": "", "description": ""}, {"column": "75%", "dtype": "number", "std": 20.392959569420032, "min": 77.05, "max": 105.89, "num_unique_values": 2, "samples": [77.05, 105.89], "semantic_type": "", "description": ""}, {"column": "max", "dtype": "number", "std": 16663.176359627538, "min": 2125.87, "max": 25691.16, "num_unique_values": 2, "samples": [2125.87, 25691.16], "semantic_type": "", "description": ""}]
{"type": "dataframe"}

```

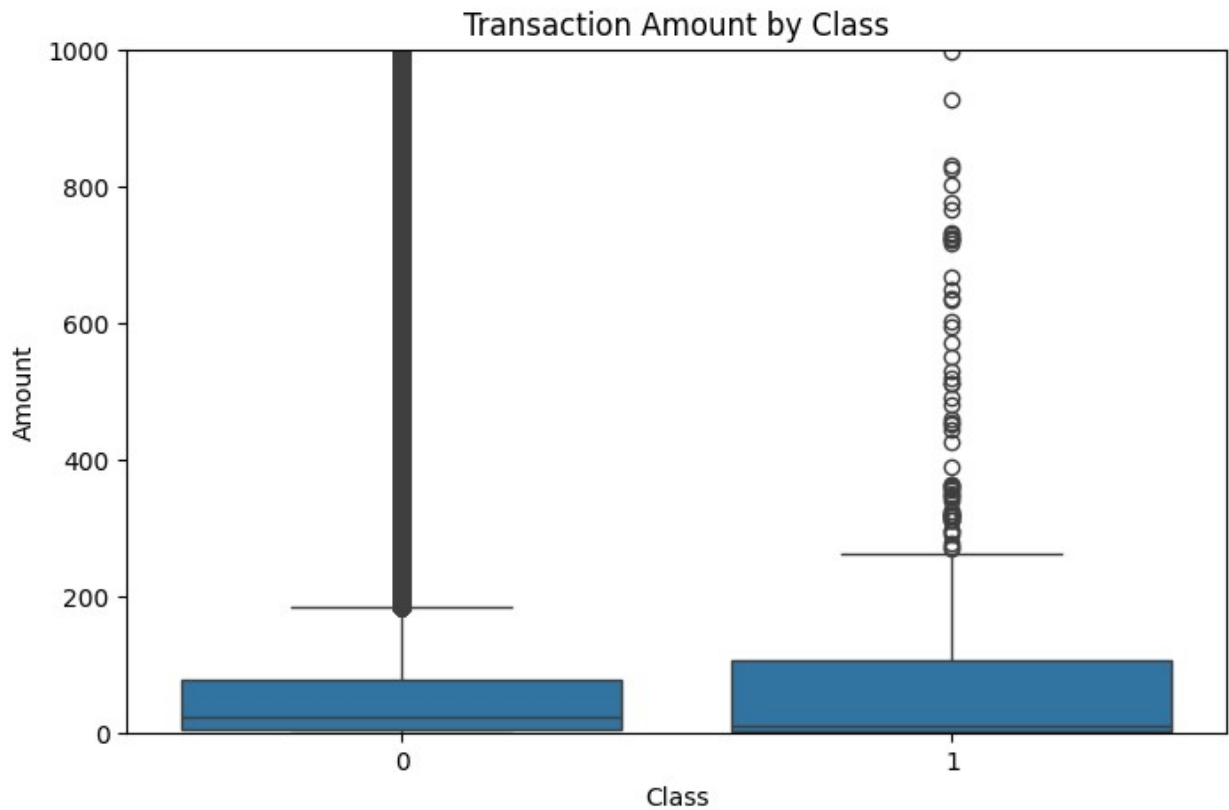
```
plt.figure(figsize=(8, 5))
```

```

sns.boxplot(
    data=df,
    x="Class",
    y="Amount"
)

```

```
plt.ylim(0, 1000)
plt.title("Transaction Amount by Class")
plt.show()
```



Analyze transaction time

```
df["Time_hours"] = df["Time"] / 3600
df["Time_days"] = df["Time"] / (3600 * 24)
df["Hour"] = (
    (df["Time"] / 3600) % 24
)
hour_fraud = (
    df.groupby("Hour")["Class"]
    .mean()
)
plt.figure(figsize=(10, 5))
```

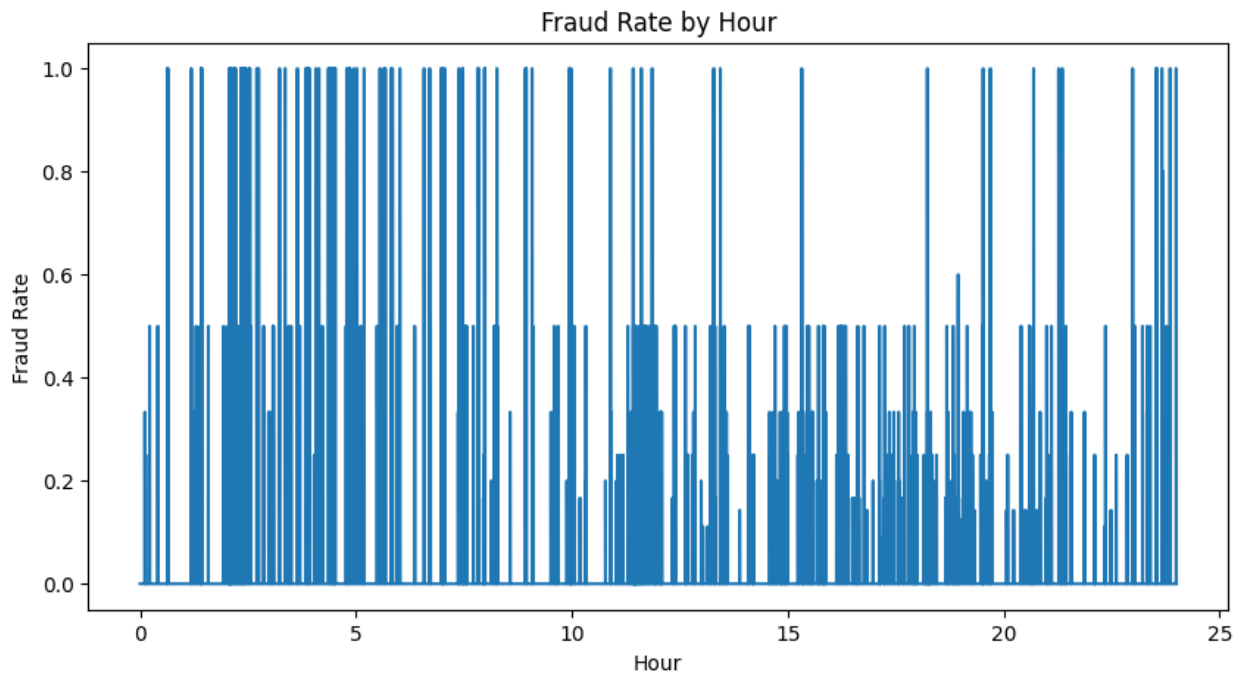
```

hour_fraud.plot()

plt.title("Fraud Rate by Hour")
plt.xlabel("Hour")
plt.ylabel("Fraud Rate")

plt.show()

```



Feature engineering

```

df["Amount_log"] = np.log1p(df["Amount"])
df["Hour"]

```

0	0.000000
1	0.000000
2	0.000278
3	0.000278
4	0.000556
...	
284802	23.996111
284803	23.996389
284804	23.996667
284805	23.996667
284806	23.997778

Name: Hour, Length: 284807, dtype: float64

```
df["Night"] = (  
    (df["Hour"] < 6) |  
    (df["Hour"] >= 22)  
) .astype(int)
```

Create amount-based features

```
amount_median = df["Amount"].median()  
  
df["Amount_to_Median"] = (  
    df["Amount"] / (amount_median + 1e-6)  
)  
  
amount_95 = df["Amount"].quantile(0.95)  
  
df["High_Amount"] = (  
    df["Amount"] > amount_95  
) .astype(int)
```

Feature engineering with PCA variables

```
v_columns = [f"V{i}" for i in range(1, 29)]  
  
df["PCA_Magnitude"] = np.sqrt(  
    (df[v_columns] ** 2).sum(axis=1)  
)  
  
df["PCA_Mean"] = df[v_columns].mean(axis=1)  
  
df["PCA_Std"] = df[v_columns].std(axis=1)  
  
df["PCA_Max"] = df[v_columns].max(axis=1)  
  
df["PCA_Min"] = df[v_columns].min(axis=1)
```

Check your engineered features

```
engineered_features = [  
    "Amount_log",  
    "Hour",  
    "Night",  
    "Amount_to_Median",  
    "High_Amount",  
    "PCA_Magnitude",  
    "PCA_Mean",
```

```

    "PCA_Std",
    "PCA_Max",
    "PCA_Min"
]

df[engineered_features].describe()

{"summary":{"\n  \"name\": \"df[engineered_features]\", \"rows\": 8, \"fields\": [\n    {\n      \"column\": \"Amount_log\", \"properties\": {\n        \"dtype\": \"number\", \"std\": 100693.25099381924, \"min\": 0.0, \"max\": 284807.0, \"num_unique_values\": 8, \"samples\": [\n          3.152188061233118, 3.1354942159291497, 284807.0\n        ], \"semantic_type\": \"\", \"description\": \"\"\n      }, {\n        \"column\": \"Hour\", \"properties\": {\n          \"dtype\": \"number\", \"std\": 100689.96929027072, \"min\": 0.0, \"max\": 284807.0, \"num_unique_values\": 8, \"samples\": [\n            14.537950752614933, 15.010833333333333, 284807.0\n          ], \"semantic_type\": \"\", \"description\": \"\"\n        }, {\n          \"column\": \"Night\", \"properties\": {\n            \"dtype\": \"number\", \"std\": 100694.40182264331, \"min\": 0.0, \"max\": 284807.0, \"num_unique_values\": 5, \"samples\": [\n              0.17665647262883286, 1.0, 0.3813783869103999\n            ], \"semantic_type\": \"\", \"description\": \"\"\n          }, {\n            \"column\": \"Amount_to_Median\", \"properties\": {\n              \"dtype\": \"number\", \"std\": 100635.30609903955, \"min\": 0.0, \"max\": 284807.0, \"num_unique_values\": 8, \"samples\": [\n                4.015891601592714, 0.9999999545454565, 284807.0\n              ], \"semantic_type\": \"\", \"description\": \"\"\n            }, {\n              \"column\": \"High_Amount\", \"properties\": {\n                \"dtype\": \"number\", \"std\": 100694.41647890914, \"min\": 0.0, \"max\": 284807.0, \"num_unique_values\": 5, \"samples\": [\n                  0.04997068190037464, 1.0, 0.2178847850090369\n                ], \"semantic_type\": \"\", \"description\": \"\"\n              }, {\n                \"column\": \"PCA_Magnitude\", \"properties\": {\n                  \"dtype\": \"number\", \"std\": 100682.7597336246, \"min\": 2.0556315559855527, \"max\": 284807.0, \"num_unique_values\": 8, \"samples\": [\n                    4.780735902632991, 4.258677975548839, 284807.0\n                  ], \"semantic_type\": \"\", \"description\": \"\"\n                }, {\n                  \"column\": \"PCA_Mean\", \"properties\": {\n                    \"dtype\": \"number\", \"std\": 100694.7858818367, \"min\": -7.084103670948337, \"max\": 284807.0, \"num_unique_values\": 8, \"samples\": [\n                      7.661099035389284e-16, \n
```

```

0.01755846007168222,\n                284807.0\n                ],\n\n\"semantic_type\": \"\", \n                \"description\": \"\"\n                }\n\n    },\n    {\n        \"column\": \"PCA_Std\", \n        \"properties\":\n        {\n            \"dtype\": \"number\", \n            \"std\":\n            100692.23046176875,\n            \"min\": 0.3901822707519061,\n            \"max\": 284807.0,\n            \"num_unique_values\": 8,\n            \"samples\": [\n                0.9057722176985058,\n                0.8080493659211809,\n                284807.0\n            ],\n            \"semantic_type\": \"\", \n            \"description\": \"\"\n            }\n        },\n        {\n            \"column\": \"PCA_Max\", \n            \"properties\":\n            {\n                \"dtype\": \"number\", \n                \"std\":\n                100687.92511191928,\n                \"min\": 0.396917304266303,\n                \"max\": 284807.0,\n                \"num_unique_values\": 8,\n                \"samples\": [\n                    2.117727397820024,\n                    1.8835522911031,\n                    284807.0\n                ],\n                \"semantic_type\": \"\", \n                \"description\": \"\"\n                }\n            },\n            {\n                \"column\": \"PCA_Min\", \n                \"properties\":\n                {\n                    \"dtype\": \"number\", \n                    \"std\":\n                    100700.53586815146,\n                    \"min\": -113.743306711146,\n                    \"max\": 284807.0,\n                    \"num_unique_values\": 8,\n                    \"samples\": [\n                        -2.050287721496829,\n                        1.72523537722886,\n                        284807.0\n                    ],\n                    \"semantic_type\": \"\", \n                    \"description\": \"\"\n                    }\n                }\n            }\n        ],\n        \"type\": \"dataframe\"}

```

SQL analysis

```

import sqlite3

conn = sqlite3.connect("/content/transactions.db")

df.to_sql(
    "transactions",
    conn,
    if_exists="replace",
    index=False
)

284807

query = """
SELECT
    Class,
    COUNT(*) AS transaction_count,
    AVG(Amount) AS average_amount,
    MAX(Amount) AS maximum_amount
FROM transactions
GROUP BY Class;

```

```

"""

sql_result = pd.read_sql(query, conn)

sql_result

{"summary":{"\n  \"name\": \"sql_result\",\n  \"rows\": 2,\n  \"fields\": [\n    {\n      \"column\": \"Class\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          1,\n          0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"transaction_count\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 200693,\n        \"min\": 492,\n        \"max\": 284315,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          492,\n          284315\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"average_amount\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 23.985273241884894,\n        \"min\": 88.29102242231328,\n        \"max\": 122.21132113821139,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          122.21132113821139,\n          88.29102242231328\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"maximum_amount\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 16663.176359627538,\n        \"min\": 2125.87,\n        \"max\": 25691.16,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          2125.87,\n          25691.16\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"sql_result\"}

```

SQL fraud-rate analysis

```

query = """
SELECT
    Hour,
    COUNT(*) AS total_transactions,
    SUM(Class) AS fraud_transactions,
    AVG(Class) AS fraud_rate
FROM transactions
GROUP BY Hour
ORDER BY Hour;
"""

hour_analysis = pd.read_sql(query, conn)

hour_analysis.head()

{"type": "dataframe", "variable_name": "hour_analysis"}

```

Prepare the ML dataset

```
features = (
    v_columns
    + [
        "Time",
        "Amount",
        "Amount_log",
        "Hour",
        "Night",
        "Amount_to_Median",
        "High_Amount",
        "PCA_Magnitude",
        "PCA_Mean",
        "PCA_Std",
        "PCA_Max",
        "PCA_Min"
    ]
)
X = df[features]
y = df["Class"]
```

Train/test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training:", X_train.shape)
print("Testing:", X_test.shape)

print(
    "Fraud rate in training:",
    y_train.mean()
)

print(
    "Fraud rate in testing:",
    y_test.mean()
)
```

```
Training: (227845, 40)
Testing: (56962, 40)
Fraud rate in training: 0.001729245759178389
Fraud rate in testing: 0.0017204452090867595
```

Logistic Regression

```
logistic_model = Pipeline([
    ("scaler", StandardScaler()),
    (
        "model",
        LogisticRegression(
            class_weight="balanced",
            max_iter=1000,
            random_state=42
        )
    )
])

logistic_model.fit(
    X_train,
    y_train
)

Pipeline(steps=[('scaler', StandardScaler()),
                 ('model', LogisticRegression(class_weight='balanced',
max_iter=1000,
random_state=42))])

lr_probability = (
    logistic_model
    .predict_proba(X_test)[: , 1]
)

lr_prediction = (
    lr_probability >= 0.5
).astype(int)
```

Random Forest

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
```

```

)

rf_model.fit(
    X_train,
    y_train
)

RandomForestClassifier(class_weight='balanced', max_depth=12,
n_estimators=200,
                        n_jobs=-1, random_state=42)

rf_probability = (
    rf_model
    .predict_proba(X_test)[:, 1]
)

rf_prediction = (
    rf_probability >= 0.5
).astype(int)

```

Gradient Boosting

```

gb_model = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

gb_model.fit(
    X_train,
    y_train
)

GradientBoostingClassifier(learning_rate=0.05, n_estimators=200,
                            random_state=42)

gb_probability = (
    gb_model
    .predict_proba(X_test)[:, 1]
)

gb_prediction = (
    gb_probability >= 0.5
).astype(int)

```

Evaluate correctly

```
def evaluate_model(name, y_true, predictions, probabilities):  
    return {  
        "Model": name,  
        "Precision": precision_score(  
            y_true,  
            predictions,  
            zero_division=0  
        ),  
        "Recall": recall_score(  
            y_true,  
            predictions,  
            zero_division=0  
        ),  
        "F1": f1_score(  
            y_true,  
            predictions,  
            zero_division=0  
        ),  
        "ROC_AUC": roc_auc_score(  
            y_true,  
            probabilities  
        ),  
        "AUPRC": average_precision_score(  
            y_true,  
            probabilities  
        )  
    }  
  
results = pd.DataFrame([  
    evaluate_model(  
        "Logistic Regression",  
        y_test,  
        lr_prediction,  
        lr_probability  
    ),  
    evaluate_model(  
        "Random Forest",  
        y_test,  
        rf_prediction,  
        rf_probability  
    ),  
    evaluate_model(  
        "Gradient Boosting",  
        y_test,  
        gb_prediction,
```

```

        gb_probability
    )
1)

results

{"summary":{"\n  \"name\": \"results\", \n  \"rows\": 3, \n  \"fields\": [\n    {\n      \"column\": \"Model\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 3, \n        \"samples\": [\n          \"Logistic Regression\", \n          \"Random Forest\", \n          \"Gradient Boosting\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    }, \n    {\n      \"column\": \"Precision\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.42752592485126667, \n        \"min\": 0.056004978220286245, \n        \"max\": 0.8181818181818182, \n        \"num_unique_values\": 3, \n        \"samples\": [\n          0.056004978220286245, \n          0.8181818181818182, \n          0.7727272727272727 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    }, \n    {\n      \"column\": \"Recall\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.04601275257982545, \n        \"min\": 0.826530612244898, \n        \"max\": 0.9183673469387755, \n        \"num_unique_values\": 3, \n        \"samples\": [\n          0.9183673469387755, \n          0.826530612244898, \n          0.8673469387755102 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    }, \n    {\n      \"column\": \"F1\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.41237980875374586, \n        \"min\": 0.10557184750733138, \n        \"max\": 0.8223350253807107, \n        \"num_unique_values\": 3, \n        \"samples\": [\n          0.10557184750733138, \n          0.8223350253807107, \n          0.8173076923076923 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    }, \n    {\n      \"column\": \"ROC_AUC\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.004747532930184787, \n        \"min\": 0.9670314348305445, \n        \"max\": 0.9762657841696047, \n        \"num_unique_values\": 3, \n        \"samples\": [\n          0.9735623234240235, \n          0.9762657841696047, \n          0.9670314348305445 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    }, \n    {\n      \"column\": \"AUPRC\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.07092103657125068, \n        \"min\": 0.6740506163362822, \n        \"max\": 0.814690940961025, \n        \"num_unique_values\": 3, \n        \"samples\": [\n          0.7284144444171567, \n          0.814690940961025, \n          0.6740506163362822 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    } \n  ] \n}, \"type\": \"dataframe\", \"variable_name\": \"results\"}

```

Precision-Recall curves

```
lr_precision, lr_recall, _ = precision_recall_curve(
    y_test,
    lr_probability
)

rf_precision, rf_recall, _ = precision_recall_curve(
    y_test,
    rf_probability
)

gb_precision, gb_recall, _ = precision_recall_curve(
    y_test,
    gb_probability
)

plt.figure(figsize=(9, 6))

plt.plot(
    lr_recall,
    lr_precision,
    label="Logistic Regression"
)

plt.plot(
    rf_recall,
    rf_precision,
    label="Random Forest"
)

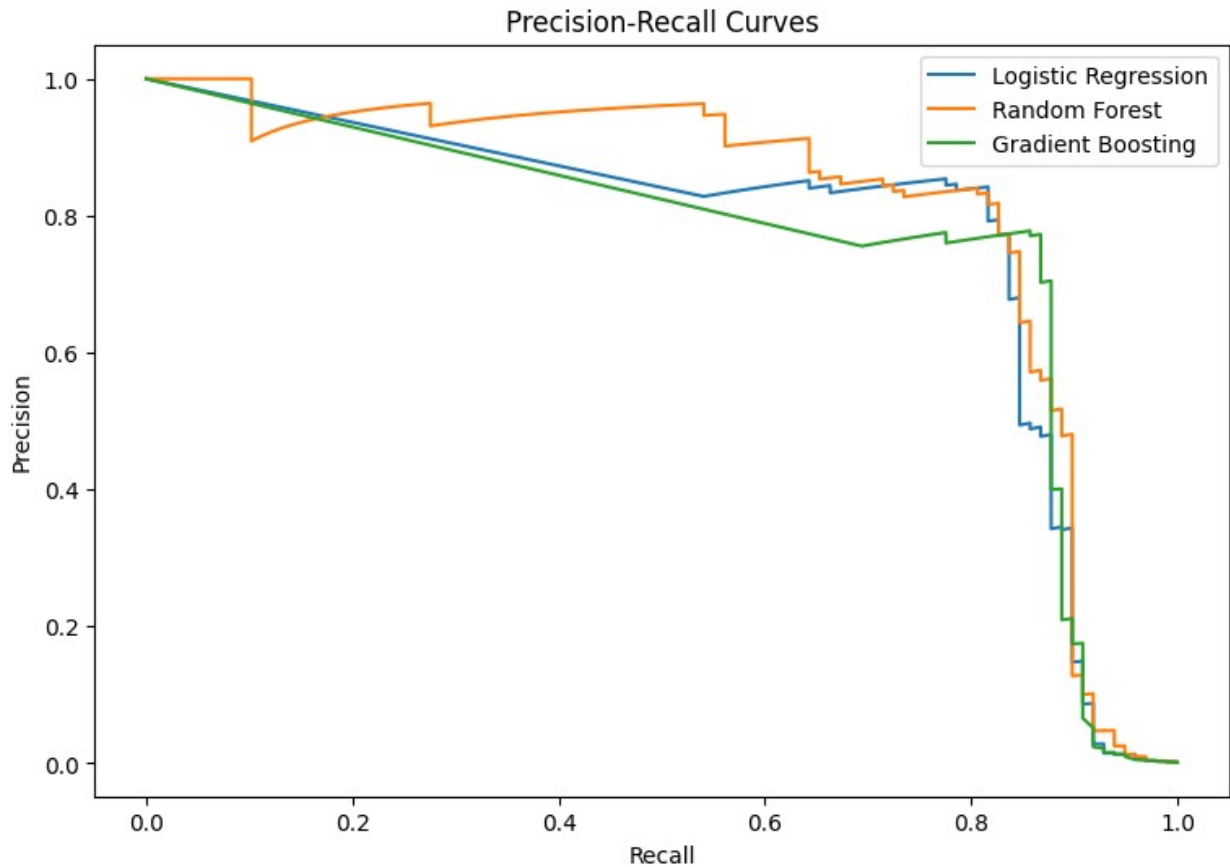
plt.plot(
    gb_recall,
    gb_precision,
    label="Gradient Boosting"
)

plt.xlabel("Recall")
plt.ylabel("Precision")

plt.title("Precision-Recall Curves")

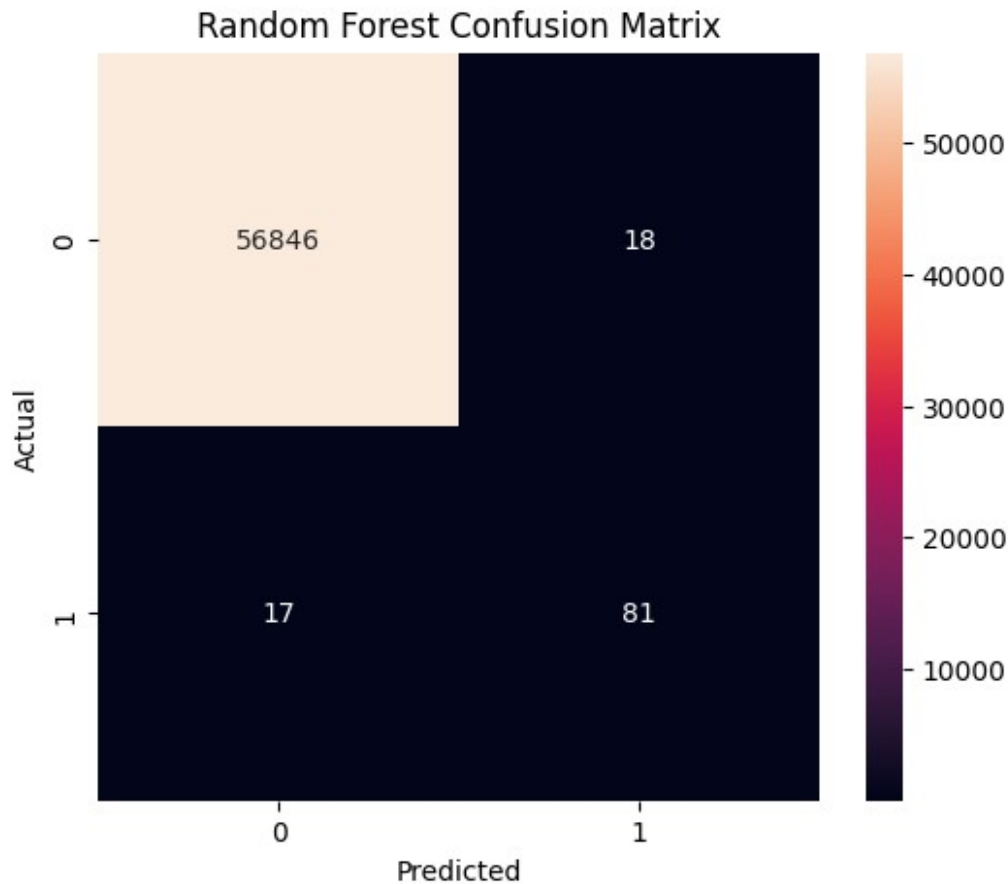
plt.legend()

plt.show()
```



Confusion matrix

```
cm = confusion_matrix(  
    y_test,  
    rf_prediction  
)  
  
plt.figure(figsize=(6, 5))  
  
sns.heatmap(  
    cm,  
    annot=True,  
    fmt="d"  
)  
  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
  
plt.title("Random Forest Confusion Matrix")  
  
plt.show()
```



Feature importance

```
feature_importance = pd.DataFrame({
    "Feature": features,
    "Importance": rf_model.feature_importances_
})

feature_importance = (
    feature_importance
    .sort_values(
        "Importance",
        ascending=False
    )
)

feature_importance.head(15)

{"summary": "{\n  \"name\": \"feature_importance\",\n  \"rows\": 40,\n  \"fields\": [\n    {\n      \"column\": \"Feature\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 40,\n        \"samples\": [\n
```

```

{"V21": {"semantic_type": "V21", "description": "V21", "importance": 0.03517932223822015, "min": 0.00024652714040388813, "max": 0.1723466864211139, "num_unique_values": 40, "samples": [0.007746917572024207, 0.008983971801576602, 0.009039880924866715]}, {"V27": {"semantic_type": "V27", "description": "V27", "importance": 0.03517932223822015, "min": 0.00024652714040388813, "max": 0.1723466864211139, "num_unique_values": 40, "samples": [0.007746917572024207, 0.008983971801576602, 0.009039880924866715]}, {"V9": {"semantic_type": "V9", "description": "V9", "importance": 0.03517932223822015, "min": 0.00024652714040388813, "max": 0.1723466864211139, "num_unique_values": 40, "samples": [0.007746917572024207, 0.008983971801576602, 0.009039880924866715]}], "type": "dataframe", "variable_name": "feature_importance"}

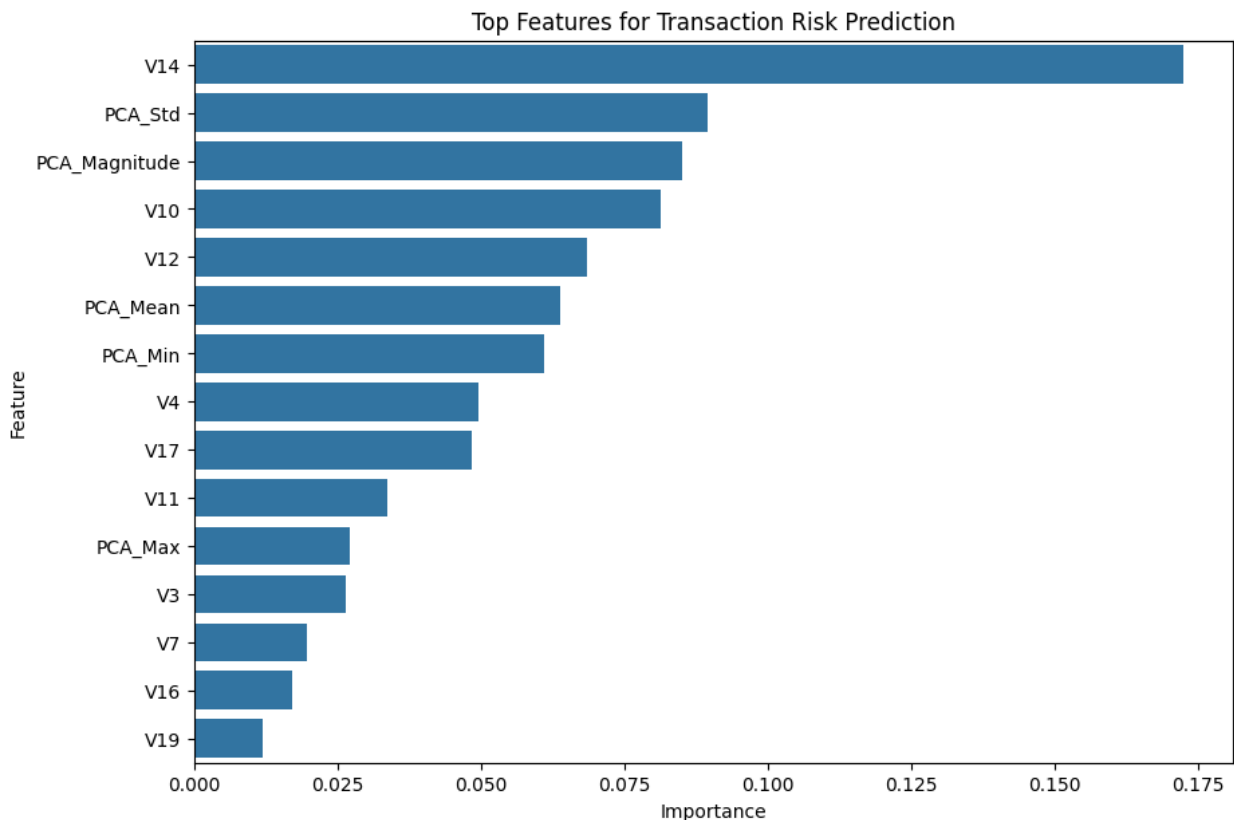
plt.figure(figsize=(10, 7))

sns.barplot(
    data=feature_importance.head(15),
    x="Importance",
    y="Feature"
)

plt.title(
    "Top Features for Transaction Risk Prediction"
)

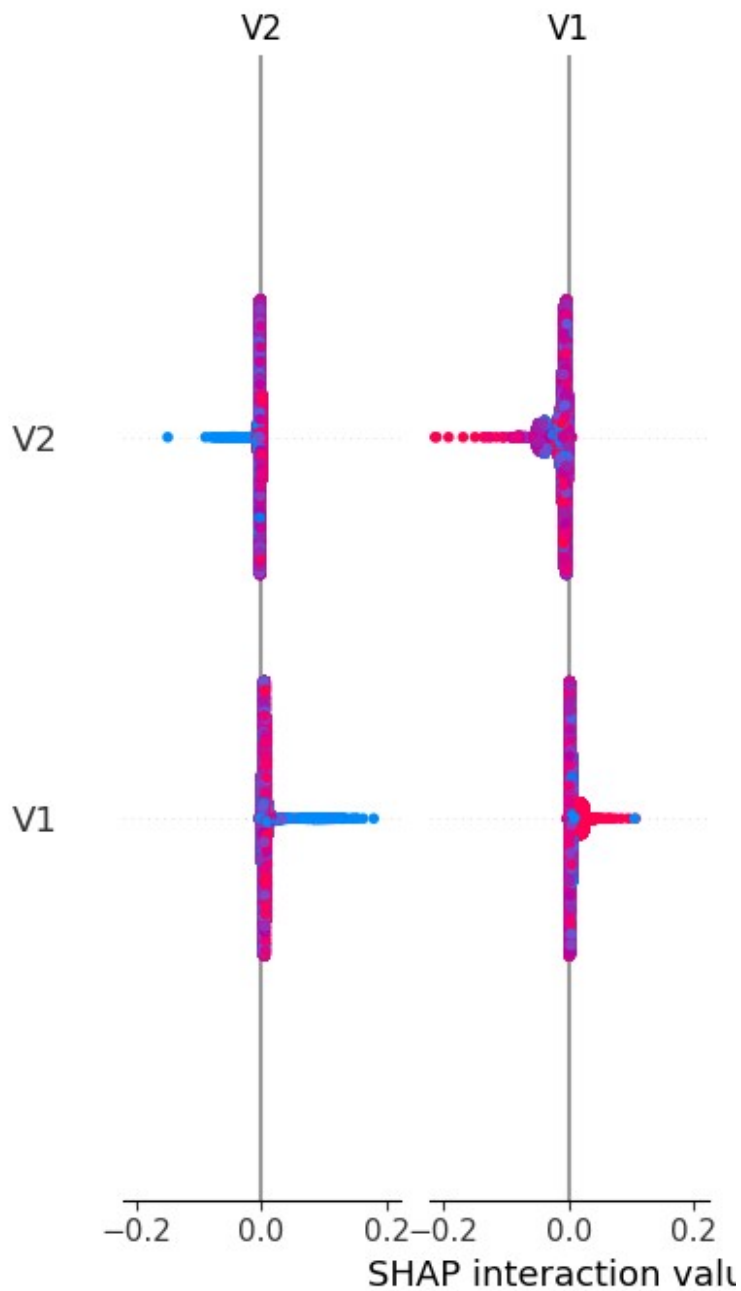
plt.show()

```



SHAP

```
explainer = shap.TreeExplainer(  
    rf_model  
)  
  
shap_values = explainer.shap_values(  
    X_test  
)  
  
type(shap_values)  
numpy.ndarray  
  
type(shap_values)  
numpy.ndarray  
  
shap.summary_plot(  
    shap_values,  
    X_test  
)
```



Create a risk score

```
risk_score = rf_probability * 100  
risk_results = X_test.copy()  
risk_results["Actual_Class"] = y_test.values  
risk_results["Risk_Score"] = risk_score
```

```
risk_results["Risk_Level"] = pd.cut(
    risk_results["Risk_Score"],
    bins=[0, 30, 70, 100],
    labels=[
        "Low",
        "Medium",
        "High"
    ],
    include_lowest=True
)
```

Create a transaction-risk visualization

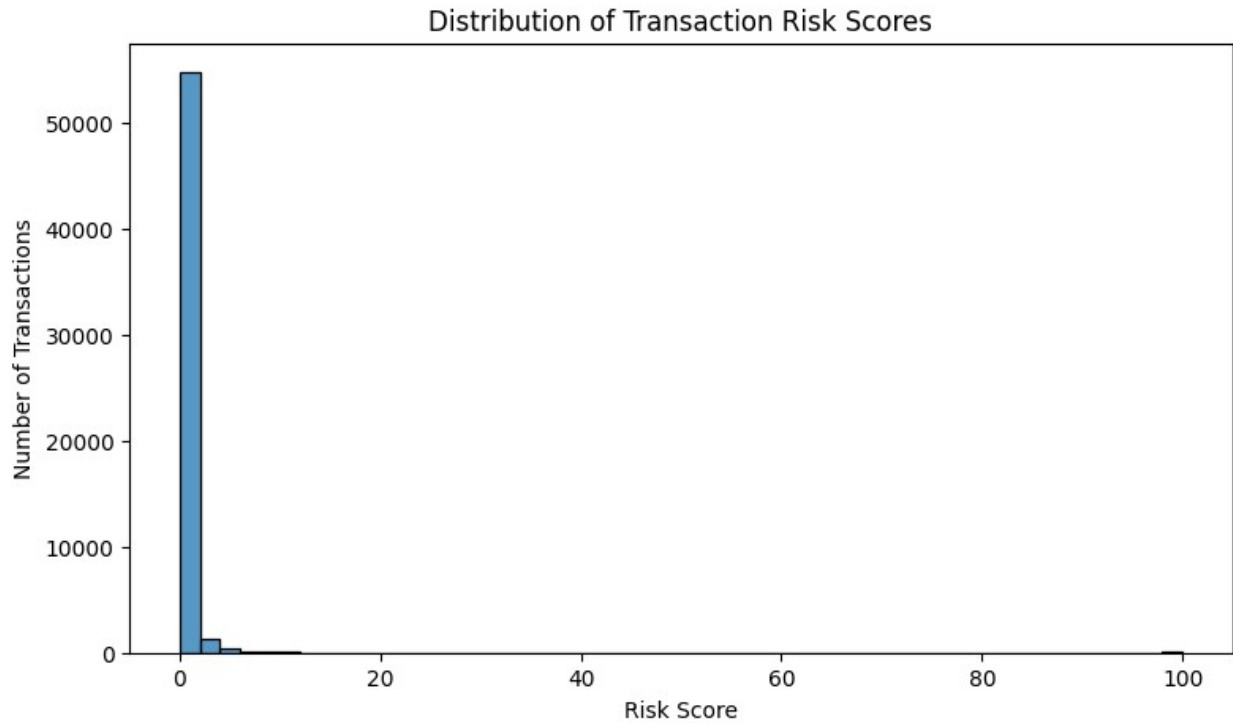
```
plt.figure(figsize=(9, 5))

sns.histplot(
    risk_results["Risk_Score"],
    bins=50
)

plt.xlabel("Risk Score")
plt.ylabel("Number of Transactions")

plt.title(
    "Distribution of Transaction Risk Scores"
)

plt.show()
```



Create a high-risk transaction table

```
high_risk = (  
    risk_results[  
        risk_results["Risk_Level"] == "High"  
    ]  
    .sort_values(  
        "Risk_Score",  
        ascending=False  
    )  
)  
  
high_risk.head(20)  
  
{"type": "dataframe", "variable_name": "high_risk"}
```